# Take-Home: Build an AI Engineering Skill

AI-assisted development is becoming a normal part of software engineering. Rather than testing how well you can code without AI, we want to understand how effectively you can use AI to improve the way an engineering organization builds software.

## The Challenge

Design and build an **AI engineering skill** that could be used across an engineering organization throughout the software development lifecycle.

The goal is not simply to create a prompt that generates code.

Build something that an engineer could realistically use as part of their day-to-day workflow to make development **faster, safer, more consistent, or higher quality**.

Your skill could address one or several parts of the engineering lifecycle—for example:

- Understanding an unfamiliar codebase
- Planning a feature
- Architecture/design decisions
- Implementing features
- Database/schema changes
- Writing or improving tests
- Code review
- Debugging
- Security review
- Documentation
- Refactoring
- Deployment/release readiness

You decide what problem is worth solving and how the skill should work.

## Requirements

Assume the skill will operate inside a real engineering organization with:

- Multiple repositories and services
- Existing architectural patterns and conventions
- Different levels of engineering experience
- CI/CD pipelines
- Production systems
- Security and reliability requirements
- Engineers using AI coding tools as part of their normal workflow

Your solution should demonstrate how the skill:

1. Understands enough context about the codebase and organization to make useful decisions.
2. Gives engineers repeatable results rather than relying on a giant one-off prompt.
3. Knows when to generate code versus when to ask questions, investigate, or refuse to proceed.
4. Validates its own work where possible.
5. Avoids introducing obvious security, architectural, or maintainability problems.
6. Can evolve as the engineering organization and codebase change.

You may use any AI models, coding agents, tools, frameworks, MCP servers, APIs, or other technologies you think are appropriate.

There are intentionally very few restrictions.

## What to Submit

Provide:

### 1. The working skill

Something we can actually run or demonstrate.

### 2. A README

Explain:

- What problem you chose to solve
- How the skill works
- How an engineer would use it
- What context/data it needs
- Key architectural decisions
- Limitations and failure modes

### 3. Example

Show the skill operating on at least one realistic engineering task.

Include the input/task, what the skill did, and the resulting output.

### 4. Reflection

Briefly answer:

> If you had to deploy this tomorrow for 20 engineers and trust the code it helps produce in production, what would you change?

## What We're Evaluating

We are not measuring how much code you personally typed versus how much AI generated.

Use AI aggressively.

We are interested in your ability to make AI useful.

We will primarily evaluate:

- Engineering judgment
- AI/tool orchestration
- Context engineering
- System design
- Developer experience
- Validation and guardrails
- Ability to recognize failure modes
- Simplicity versus unnecessary complexity
- Whether engineers would actually want to use what you built

Be prepared to walk us through your solution, including what AI generated, what decisions you made yourself, what didn't work, and what you would improve.